AMTE SecAI

بِسۡمِ ٱللَّهِ ٱلرَّحۡمَٰنِ ٱلرَّحِيمِ

Detailed Table of Contents

Chapter	Title
Chapter 1	**Introduction and Objectives**
1.1	Project Overview (Introduction)
1.2	Problem Statement
1.3	About the Team
1.4	Project Objectives (Technical and Functional)
1.5	Significance of the Project
1.6	Project Scope and Limitations
Chapter 2	**Literature Review**
2.1	Introduction
2.2	The Evolution from Traditional to AI-Driven Security
2.3	Machine Learning Applications in Cyber Threat Detection
2.4	Review of Related Work and Systems
2.5	Existing System Analysis and the Research Gap
Chapter 3	**User Requirements & System Requirements Specification (SRS)**
3.1	Introduction to Requirements Engineering
3.2	User Requirements (High-Level Needs)
3.2.1	Dataset Upload Requirement (CSV / JSONL)
3.2.2	Kaggle Dataset Integration Requirement
3.2.3	Data Cleaning & Icon/Noise Removal
3.2.4	Data Visualization Requirement
3.2.5	Machine Learning Model Training Requirement
3.2.6	Performance & Speed Requirement (GPU Support)
3.3	Functional Requirements (System Shall Statements)
3.3.1	Data Loading & Parsing

3.3.2	Preprocessing and Filtering
3.3.3	Feature Extraction
3.3.4	Model Training & Testing
3.3.5	Result Visualization
3.3.6	Exporting Cleaned Data
3.4	Non-Functional Requirements
3.4.1	Performance
3.4.2	Reliability
3.4.3	Scalability
3.4.4	Security
3.4.5	Usability
3.4.6	Compatibility
3.5	System Constraints & Assumptions
Chapter 4	**System Analysis**
4.1	System Architecture Overview
4.2	Workflow Diagram
4.3	Data Flow Diagram (DFD)
4.4	Use Case Diagram
4.5	Hardware Requirements
4.6	Software Requirements
Chapter 5	**System Design**
5.1	High-Level Design (HLD)
5.2	Low-Level Design (LLD)
5.3	Dataset / Database Structure Design
5.4	Module Design
5.5	Algorithm Design
5.6	GPU Acceleration Design
Chapter 6	**Machine Learning Model**
6.1	Data Acquisition from Kaggle and Other Sources
6.2	Data Preprocessing and Filtering

6.3	Feature Selection and Engineering
6.4	Model Architecture
6.5	Model Training Process
6.6	Model Validation and Testing Methods
6.7	Evaluation Metrics (Accuracy, Precision, Recall, F1-score)
Chapter 7	**System Implementation**
7.1	Technologies Used (Python / C++ / CUDA / etc.)
7.2	Dataset Integration Implementation
7.3	Preprocessing Module Implementation
7.4	Model Training Implementation
7.5	Visualization Module Implementation
7.6	GPU Optimization Implementation
7.7	User Interface Implementation
Chapter 8	**Results, Testing & Discussion**
8.1	Test Cases and Experimental Setup
8.2	Performance Analysis
8.3	Graphs and Visualization Results
8.4	Comparative Analysis
8.5	Discussion of Results
Chapter 9	**Challenges, Errors, and Hardships Faced**
9.1	Feasibility and Research Validation Issues
9.2	Data Sourcing and Quality Problems
9.3	Big Data Handling Challenges
9.4	Hardware and GPU Constraints
9.5	Optimization and Algorithm Selection Issues
9.6	Debugging & Error Handling
Chapter 10	**Conclusion & Future Work**
10.1	Project Summary
10.2	Major Contributions
10.3	Final Conclusion

10.4	Limitations of the Current System
10.5	Future Work and Recommendations

Project Overview (Introduction)

In an era defined by the increasing sophistication of cyberattacks, relying solely on traditional, signature-based security solutions is no longer sufficient to protect individual devices. Malware and threats are evolving at an unprecedented pace, demanding a defensive system that is intelligent, adaptive, and predictive.

Our graduation project, **AMTE SecAI (Advanced Machine Learning Threat Evasion System)**, is a direct response to this challenge. We have developed a powerful and innovative **Desktop Application** that integrates cutting-edge **Artificial Intelligence (AI)** and **Machine Learning (ML)** technologies to significantly elevate the security posture of user devices.

The core objective of **AMTE SecAI** is to **strengthen the security of the desktop device**, making it more robust and effective against persistent and unknown threats. The application functions as a proactive monitor and intelligent analyzer, focusing on the following key areas:

- **Intelligent Threat Detection:** Utilizes sophisticated Machine Learning algorithms to analyze system behavior, network traffic, and process activity in real-time.
- **Instant Alerting:** Provides **immediate and clear alerts** upon the detection of any suspicious or abnormal activity, such as potential hacking attempts or ransomware activity, ensuring the user is notified instantly of any security compromise.
- **Gmail Spam/Phishing Check:** Includes a critical added feature that analyzes incoming **Gmail** messages to accurately identify **Phishing** attempts or **Spam**, thereby protecting the user from social engineering and email-based scams.

In summary, **AMTE SecAI** aims to transform desktop security from a system that reacts post-incident, into an **intelligent, proactive, and predictive system** capable of anticipating threats and neutralizing them before any harm is done.

Problem Statement

The central challenge in modern cybersecurity is the time lag between the initiation of a network attack and the successful detection and mitigation of that threat. Existing security systems are often overwhelmed by the sheer volume of network traffic and fail to differentiate subtle, anomalous attack patterns from benign behavior. This failure results in a critical vulnerability: **user data remains exposed and at risk because detection and alert generation are too slow or inaccurate.** Specifically, current systems lack the necessary intelligence to predictively identify evolving threats—such as Distributed Denial of Service (DDoS) attempts, sophisticated probing, or lateral movement—with the speed and reliability required to initiate protective measures instantly. Therefore, there is a compelling need for a highly optimized, GPU-accelerated Machine Learning framework capable of processing high-volume network streams in real-time, accurately predicting attacks, and generating immediate, actionable alerts to protect the confidentiality and availability of user data before compromise occurs.

About the Team

Ahmed Rafat Fawzy:	***202202311***
Ahmed Rabeh:	***202202295***
Abd EL-Rahman EL-Nagar:	***202202252***
Eslam EL-Refaye:	***202202310***
Tariq Ahmed Motaw:	***202202304***
Mahmoud Soliman:	***202202309***

Ahmed Rafat: senior at SU, software engineer problem solver, Back-end developer ,

Ahmed Rabeh: senior at SU, security engineer

Abd EL-Rahman EL-Nagar: senior at SU, Data Analyses engineer,

Tariq Ahmed Motaw: senior at SU, Devops engineer, Back-end developer

Mahmoud Soliman: senior at SU software engineer, backend developer.

1.3 Project Objectives (Technical and Functional)

The primary goal of the AMTE SecAI project is to develop a fully functional, real-time desktop security application that demonstrates the practical superiority of Machine Learning over traditional security methods. This overarching goal is broken down into the following specific, measurable, and achievable objectives:

A. Functional Objectives (What the System Must Do)

1. **Unified Security Integration:** To successfully integrate three distinct security functions—real-time endpoint monitoring, malicious process classification, and external email threat detection—into a single, user-friendly desktop application.
2. **Proactive Threat Analysis:** To implement a core logic module capable of collecting and analyzing system data (process behavior, API calls) in real-time to identify behavioral anomalies indicative of a zero-day or fileless attack.
3. **Third-Party Integration:** To establish secure API connectivity with the Gmail service to retrieve and classify incoming email messages based on their likelihood of being spam or phishing.
4. **Instantaneous User Alerting:** To ensure the system generates immediate and clear desktop alerts upon the detection of any high-risk threat in either the endpoint or the email security module.

B. Technical Objectives (How Well the System Must Perform)

1. **High Classification Accuracy:** The developed Machine Learning models (for both endpoint processes and email classification) must achieve an **F1-Score of 95% or greater** on the relevant hold-out validation datasets.
2. **Low System Overhead:** The background monitoring service must maintain an **average CPU utilization below 5%** on the target Windows operating system during periods of system idleness to ensure a non-disruptive user experience.
3. **Robustness and Reliability:** The application must demonstrate stability and reliability by running the real-time monitoring agent continuously for a test period of at least 48 hours without crashing or requiring manual restarts.

1.4 Project Scope and Limitations

This section defines the precise boundaries and constraints under which the AMTE SecAI project is developed, ensuring clarity regarding what the project covers and what factors may limit its performance.

A. Project Scope (What is Included)

- **Target Platform:** The application is scoped exclusively for the **ALL operating systems**.
- **Security Focus:** The scope is limited to **detection and alerting** mechanisms. The system will recommend actions (terminate, quarantine) but will not implement complex automated cleanup or system restoration.
- **Email Integration:** The email analysis feature is restricted to **Gmail API integration** for classifying unread messages.
- **ML Implementation:** The project focuses on the implementation and performance evaluation of **Python-based ML models** (e.g., using TensorFlow or scikit-learn).

B. Project Limitations (Constraints)

- **Data Availability:** The final accuracy of the ML models is inherently constrained by the **size, quality, and representativeness** of the publicly available datasets used for training (e.g., malware samples, phishing email corpora).
- **Time Constraint:** Development is subject to the **academic timeline**, which may limit the opportunity for extensive feature development or long-term beta testing.
- **Resource Management:** Due to the single-user, academic nature of the project, the system will not implement cloud-based ML model retraining or centralized threat intelligence sharing, relying solely on local, static model deployment.
- **API Restriction:** Full system access (e.g., kernel-level monitoring) is restricted by operating system permissions, potentially limiting the depth of data collection compared to commercial enterprise solutions.

Literature Review

2.1 Introduction

This chapter aims to critically review and analyze the published research and existing technological solutions at the intersection of **Cybersecurity** and **Machine Learning (ML)**. This review serves two primary purposes: to establish a solid theoretical understanding of the techniques relevant to **AMTE SecAI**, and to identify the existing research gap that this project seeks to fill by delivering an integrated, intelligent desktop security system.

2.2 The Evolution from Traditional to AI-Driven Security

This section establishes the context for why ML is necessary in modern security:

2.2.1 Traditional Security Solutions (Signature-Based Detection)

- **Description:** Reliance on databases of known malware signatures and heuristics.
- **Limitations:** Inability to detect zero-day attacks, fileless malware, or polymorphic threats, leading to a reactive defense posture.

2.2.2 Behavioral and Heuristic Detection

- **Description:** Monitoring the actions and behaviors of files and processes rather than static code signatures.
- **AMTE SecAI Context:** While an improvement, conventional heuristic systems often struggle with high false positive rates, highlighting the need for the nuanced decision-making capabilities of AI.

2.3 Machine Learning Applications in Cyber Threat Detection

This section focuses on the specific ML techniques that underpin the functionalities of AMTE SecAI:

2.3.1 Malware Detection and Classification

- **Relevant Literature:** Review studies utilizing supervised classification algorithms (e.g., Support Vector Machines - SVM, Random Forest, Deep Neural Networks) for analyzing static file features (e.g., entropy, PE header analysis) or dynamic process behavior.

- **Critical Analysis:** Evaluate the trade-offs between detection accuracy, feature engineering complexity, and the real-time processing speed required for a desktop application.

2.3.2 Anomaly and Intrusion Detection Systems (AIDS)

- **Relevant Literature:** Focus on research employing unsupervised or semi-supervised ML models (e.g., Autoencoders, One-Class SVM) to establish a "baseline of normalcy" for system activity, including CPU usage, memory access, and API calls.
- **AMTE SecAI Context:** These techniques are crucial for AMTE SecAI's ability to identify previously unknown attacks by detecting deviations from the established desktop profile, forming the core of its proactive defense mechanism.

2.3.3 Phishing and Spam Email Detection

- **Relevant Literature:** Review Natural Language Processing (NLP) models, such as those based on TF-IDF feature extraction or more modern transformers, used for analyzing email content, headers, and associated URLs to classify messages as legitimate, spam, or phishing attempts.
- **Critical Analysis:** Compare the performance of different ML models in achieving high accuracy in classifying real-world, localized phishing attempts, which directly informs the design of AMTE SecAI's **Gmail Spam Check** feature.

2.4 Existing System Analysis and the Research Gap

This section justifies the need for **AMTE SecAI** by contrasting it with existing solutions:

2.4.1 Commercial Endpoint Protection Platforms (EPP)

- **Observation:** Many commercial EPPs have integrated basic ML capabilities, but they often lack transparency regarding model performance and may be resource-heavy.
- **Comparison:** AMTE SecAI aims to offer a transparent, lightweight, and focused desktop solution suitable for academic demonstration and personalized security management.

2.4.2 Current Academic Solutions

- **Observation:** Academic projects often focus deeply on a single problem (e.g., network traffic anomaly or malware classification) but fail to integrate these solutions into a unified, user-friendly desktop application.
- **The Research Gap:**

  "The review reveals a significant gap in the availability of a **single, integrated desktop application** that cohesively combines **real-time behavioral ML-based endpoint security** with a dedicated, intelligent **email security analysis module (Gmail)**. AMTE

SecAI is specifically designed to bridge this gap by delivering a unified, transparent, and proactive defense system for the end-user."

Chapter 3 — User Requirements & System Requirements Specification (SRS)

3.1 Introduction to Requirements Engineering

Requirements Engineering (RE) is the structured and systematic process of identifying, analyzing, documenting, validating, and managing the functional and non-functional requirements of a software system.

For the **AMTE SecAI system**, the goal is not only to detect cyber-attacks, but to **analyze extremely large-scale network traffic data in an efficient, optimized, and realistic environment**. Therefore, the requirements of this project extend beyond basic functionality to include **high-level performance, scalability, reliability, and GPU-based acceleration**.

This chapter defines the complete set of:

- **User Requirements** (what the user needs)
- **Functional Requirements** (what the system must do)
- **Non-Functional Requirements** (how the system must perform)
- **System Constraints and Assumptions**

These requirements are aligned with the real technical challenges faced during implementation, including large data volume, hardware limitations, and the need for high-performance processing.

3.2 User Requirements (High-Level Needs)

User Requirements describe what the user expects to achieve using the AMTE SecAI system.

3.2.1 Dataset Upload Requirement (CSV / JSONL)

The user shall be able to upload one or multiple network traffic dataset files in **CSV** and **JSONL** formats in order to perform large-scale analysis and model training on real network flow data.

3.2.2 Public Dataset Integration Requirement (CICIDS / EMBER / Kaggle)

The user shall have the ability to integrate public cybersecurity datasets such as **CICIDS 2017, EMBER**, or other Kaggle-based datasets, either through direct download or API connection, in order to benchmark and evaluate model performance on standardized data.

3.2.3 Advanced Data Cleaning & Noise Removal Requirement

The user shall be able to perform deep data cleaning operations that automatically identify and remove:

- Missing and null values
- Infinite values and corrupted rows
- Duplicate records
- Non-informative features (zero-variance columns such as *Fwd Avg Bytes/Bulk, Bwd Avg Bulk Rate, Fwd Avg Packets/Bulk*)
- Invalid labels or inconsistent formatting

This is based on the actual analysis performed by the C++/CUDA filtering program.

3.2.4 Data Analysis & Visualization Requirement

The user shall be able to generate analytical visualizations that include:

- Correlation matrices (heatmaps)
- Feature distributions (histograms)
- Class imbalance charts (label distribution)
- Scatter and density plots

These visualizations help the user understand relationships between features and identify abnormal behaviour before model training.

3.2.5 Machine Learning Model Control & Training Requirement

The user shall be able to train **two main machine learning models**:

- **Deep Neural Network (DNN)**
- **XGBoost Classifier**

The user shall also be able to adjust key hyperparameters such as:

- Epochs
- Learning rate
- Batch size
- Tree depth (XGBoost)

Training progress and performance metrics must be visible to the user.

3.2.6 High Performance & GPU Acceleration Requirement

Due to the massive size of the datasets, the user requires:

- GPU-accelerated training using **CUDA**
- Parallel data processing
- Fast feature analysis and filtering
- Optimized memory usage

The system must automatically detect and use any available **CUDA-enabled GPU** to significantly reduce execution time.

3.2.7 Real-Time Detection & Alerting Requirement

The user shall be able to use the trained model to:

- Detect suspicious or malicious network activity in real-time (or simulated live data)
- Receive immediate alerts indicating:
    - Attack type
    - Confidence level
    - Timestamp

This requirement represents the core purpose of the AMTE SecAI system.

3.3 Functional Requirements (System SHALL Statements)

These define what the system must do internally to meet the user's needs.

3.3.1 Data Loading & Parsing

The system SHALL:

- Load large CSV / JSONL datasets securely
- Support reading multiple files at once
- Convert raw data into an internal numerical structure

3.3.2 Hybrid Preprocessing & Filtering

The system SHALL apply a **hybrid filtering approach**, including:

- Fast CPU-based structural validation
- Deep GPU-based parallel processing using CUDA
- Removal of:
    - Missing values
    - Duplicates
    - Corrupted rows
    - Zero-variance columns
- Feature normalization and scaling

This requirement is based on the real implementation used in your project.

3.3.3 Feature Engineering & Optimization

The System SHALL:

- Extract the most relevant features
- Remove useless or redundant features
- Optionally apply dimensionality reduction techniques if needed
- Prepare the dataset in a format suitable for:
    - Neural Networks
    - XGBoost

3.3.4 Model Training & Evaluation

The System SHALL:

- Split the dataset into:

    - Training set
    - Validation set
    - Testing set
- Train both:
    - Deep Neural Network
    - XGBoost model
- Compute:
    - Accuracy
    - Precision
    - Recall
    - F1-Score
    - Confusion Matrix
    - ROC Curve

These metrics SHALL be stored for later analysis.

3.3.5 Result Visualization

The System SHALL visually display:

- Confusion matrix
- ROC curve
- Precision–Recall curve
- Training & validation loss curves

These visual tools SHALL assist in interpreting system performance.

3.3.6 Processed Data Export

The System SHALL allow the user to export:

- Cleaned datasets
- Normalized datasets
- Feature-selected datasets

in **CSV format** for future use or external analysis.

3.3.7 Real-Time Monitoring & Alert Generation

The System SHALL:

- Monitor live or simulated traffic
- Apply the trained model in real-time
- Generate automatic alerts when malicious activity is detected

---

3.3.8 Suspicious Repeated IP Behavior Detection (Optional SOC-Inspired Feature)

Description:
The system SHALL include an *optional security enhancement* inspired by SOC operations that analyzes repeated appearances of specific IP addresses across network flows. If an IP address exceeds a defined repetition threshold, the system SHALL automatically flag it as *Suspicious or Potential Attacker*.

Purpose:
To improve behavioral threat detection by identifying abnormal communication patterns such as:
– Botnet Command & Control communication
– DDoS repeated-source flooding
– Reconnaissance scanning attempts
– Brute-force repetitive trials

Processing Logic:

- Count per-IP occurrences across time windows
- Compare with dynamic or fixed thresholds
- Classify as:
  Normal / Warn / High-Risk (Potential Attacker)
- Integrate with real-time alert module

Notes:

- This feature does not replace core ML-based detection.
- It serves as a complementary SOC-style behavioral mechanism.
- It is optional and can be disabled by the user

3.4 Non-Functional Requirements

3.4.1 Performance

- Minimum accuracy: **≥ 95%**
- Inference time: **≤ 100 ms**
- GPU must significantly reduce training time compared to CPU-only processing

3.4.2 Reliability

- Must run for at least **72 hours continuously**
- No crashes or memory leaks
- Automatic error handling and logging

3.4.3 Scalability

- Must support datasets up to **10+ million records**
- Must handle traffic of **10,000+ packets/sec**

3.4.4 Security

- No sensitive data is transmitted externally
- Model files are protected
- User access is restricted

3.4.5 Usability

- Simple and intuitive interface
- Clear alerts and logs
- Designed for security analysts

3.4.6 Compatibility

- Supports **Windows (primary environment)**
- Compatible with CUDA-supported systems
- Can run on Linux/Mac with reduced GPU support
- Works with Python + C++ backend

3.5 System Constraints & Assumptions

This project is developed under a set of realistic constraints and assumptions that directly influence the system performance, scalability, and experimental results. These factors were identified during the implementation and testing stages of the AMTE SecAI system.

These constraints and assumptions were carefully documented to maintain scientific transparency and ensure the validity of the experimental outcomes.

3.5.1 System Constraints

Category	Constraint	Description
Data Source Limitation	Public datasets only	The system mostly depends on publicly available datasets such as **CIC-IDS 2017** , **CIC-IDS 2018** and **EMBER**. This means that certain advanced or zero-day attacks may not be fully represented.
Hardware Limitation	GPU dependency	The system's full performance (including the < 100 ms inference time) depends on the availability of a **CUDA-enabled GPU**. In CPU-only environments, execution time increases significantly.
Memory Limitation	RAM restrictions	Due to limited system RAM, large datasets could not be fully processed using traditional tools (Python only). This was the main reason behind adopting the **C++ & CUDA hybrid approach**.
Computing Environment	OS & Architecture	The system was developed and optimized for **Windows (64-bit)** environments. CUDA does not support 32-bit architecture, which created an initial configuration challenge.
Dataset Imbalance	Class distribution	Real-world network traffic is naturally imbalanced. Normal traffic vastly outnumbers attacks, which could lead to biased model predictions if not handled correctly.
Real-Time Data Access	Limited live data	The project mostly relies on stored datasets, and live traffic monitoring is simulated due to network access and privacy constraints.

3.5.2 System Assumptions

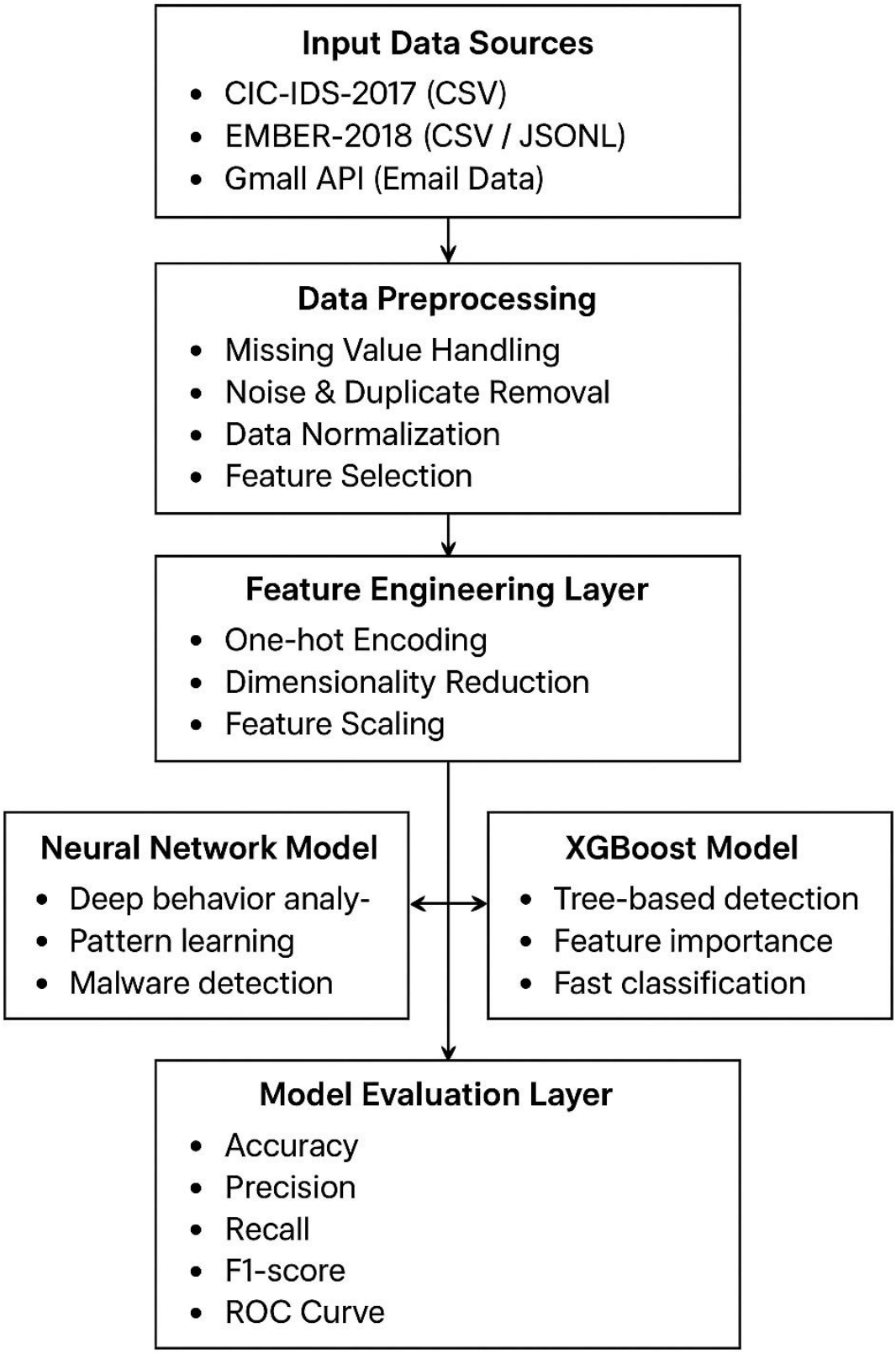
Category	Assumption	Description
Label Accuracy	Correct labeling	It is assumed that the labels in datasets such as CIC-IDS and EMBER correctly represent the true nature of the traffic (Normal, DoS, DDoS, Malware, etc.).
Feature Consistency	Stable features	It is assumed that the main features selected for model training will continue to exist and remain relevant in other environments and networks.
User Expertise	Skilled user	The system assumes the user has basic understanding of machine learning, cybersecurity, and data analysis to correctly configure and interpret the system output.
Operational Conditions	Normal network behavior	It is assumed that the real-time traffic patterns used for testing resemble the statistical behavior of the training data.
Model Generalization	Similar future attacks	The system assumes that future cyber-attacks will follow patterns somewhat related to known types; extreme novel attack types may reduce accuracy.
GPU Availability	Hardware support exists	It is assumed that the target deployment environment includes a CUDA-supported GPU to enable full system capabilities.
System Updates	Periodic retraining	The system assumes that periodic model retraining will be performed to maintain detection accuracy over time as threats evolve

Chapter 4 – System Analysis

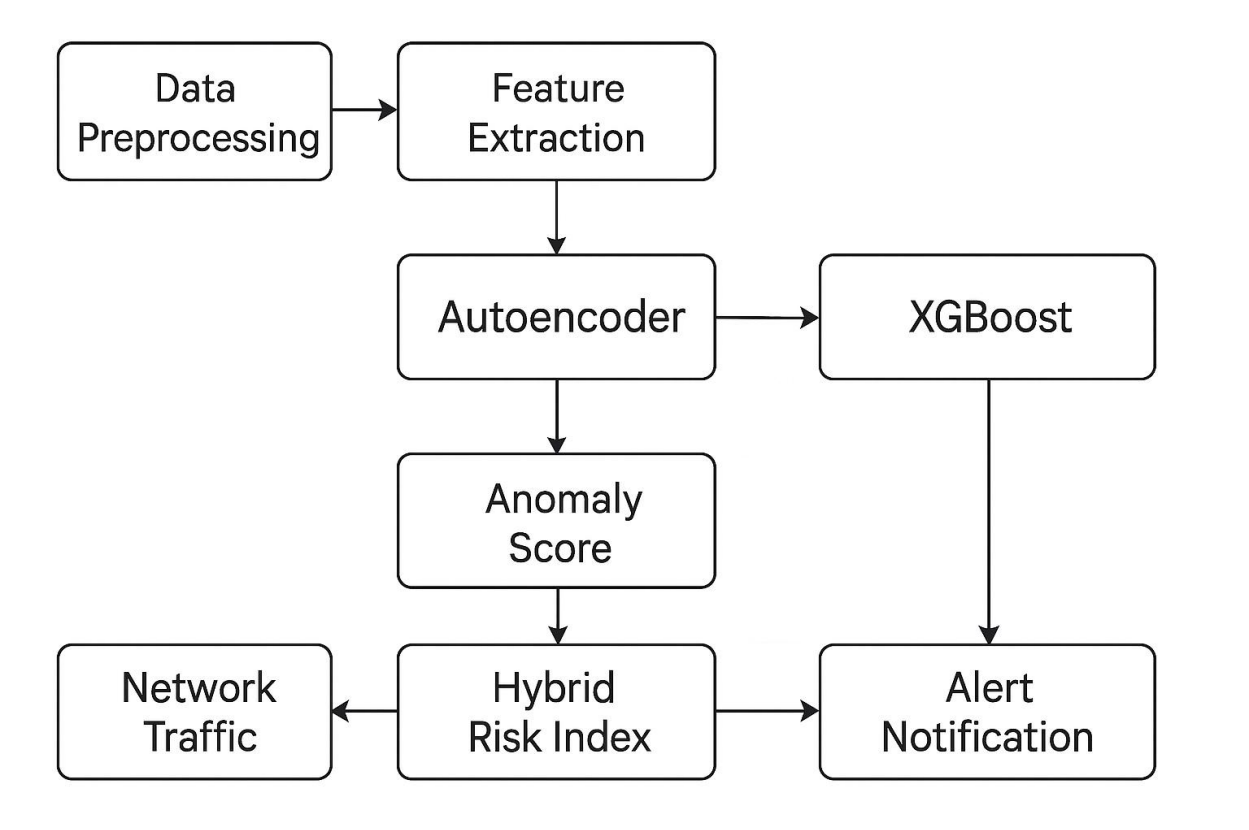
4.1 System Architecture Overview

This section presents the overall architecture of the proposed system **AMTE SecAI (Advanced Machine Learning Threat Evasion System)**. The architecture illustrates how data flows through the system starting from input, through preprocessing and model analysis, until the final detection and alerting stages.

The system is designed as a hybrid AI-based cybersecurity platform consisting of multiple integrated modules that work together to provide real-time attack detection and intelligent alerting.



Fiigure 4.1: The Overall System Architecture of



4.1.1 Overall System Architecture Diagram

```
+-------------------------------------------------------+
|                    Input Data Sources                 |
|  - CIC-IDS-2017 (CSV)                                 |
|  - EMBER-2018 (CSV / JSONL)                           |
|  - Gmail API (Email Data)                             |
+-------------------------------------------------------+
                         |
                         v
+-------------------------------------------------------+
|                    Data Preprocessing                 |
| - Missing Value Handling                              |
| - Noise & Duplicate Removal                           |
| - Data Normalization                                  |
| - Feature Selection                                   |
+-------------------------------------------------------+
                         |
                         v
+-------------------------------------------------------+
|               Feature Engineering Layer               |
| - One-hot Encoding                                    |
| - Dimensionality Reduction                            |
| - Feature Scaling                                     |
+-------------------------------------------------------+
            |                                     |
            |                                     |
            v                                     v
+------------------------------+     +-------------------------+
```

```
|      Neural Network Model    |     |      XGBoost Model      |
| - Deep behavior analysis     |     | - Tree-based detection  |
| - Pattern learning           |     | - Feature importance    |
| - Malware detection          |     | - Fast classification   |
+------------------------------+     +-------------------------+
            |                                   |
            +--------------------+---------------+
                                 |
                                 v
+-------------------------------------------------------+
|                 Model Evaluation Layer                |
|  Accuracy | Precision | Recall | F1-score | ROC Curve |
+-------------------------------------------------------+
                                 |
                                 v
+-------------------------------------------------------+
|             Real-Time Detection & Alerting            |
| - Threat level classification                         |
| - Instant user notifications                          |
| - System protection recommendations                   |
+-------------------------------------------------------+
```

Figure 4.1: The Overall System Architecture of AMTE SecAI

This architecture shows the integration of deep learning and machine learning models to strengthen threat detection capabilities while maintaining speed, accuracy, and scalability.

4.2 System Workflow Diagram

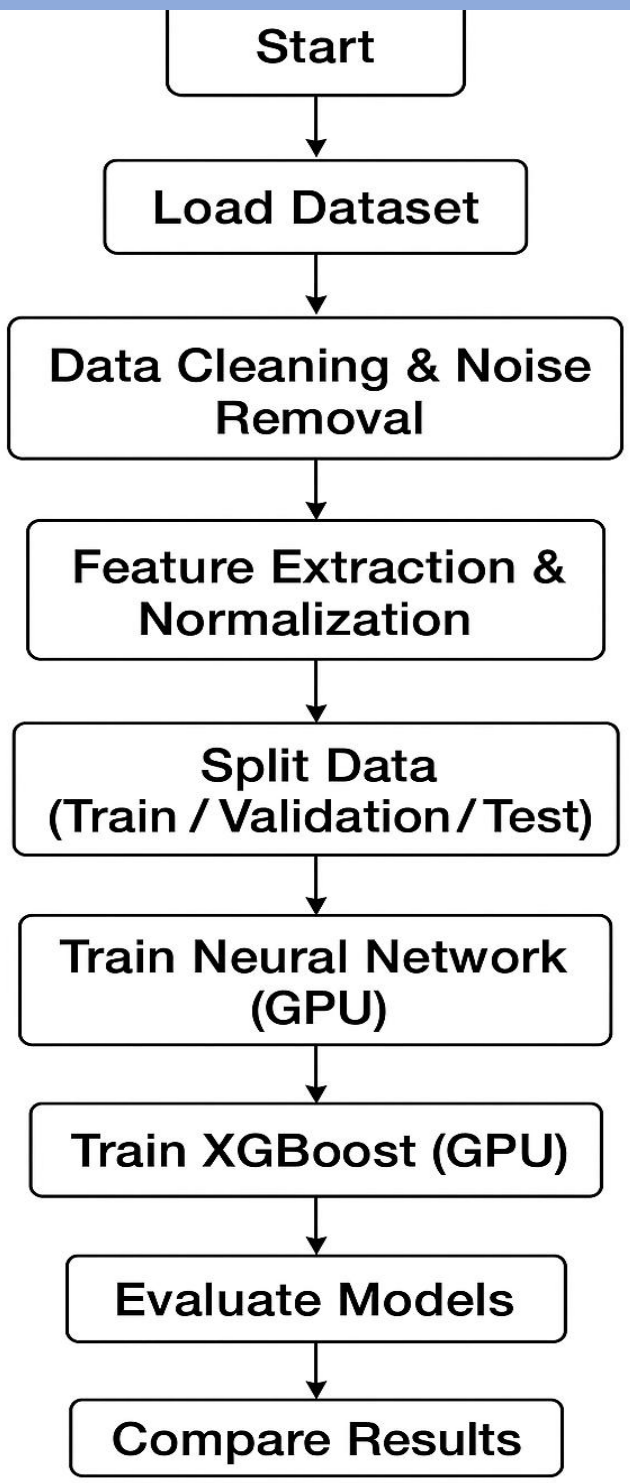


Figure 4.2: System Workflow Diagram

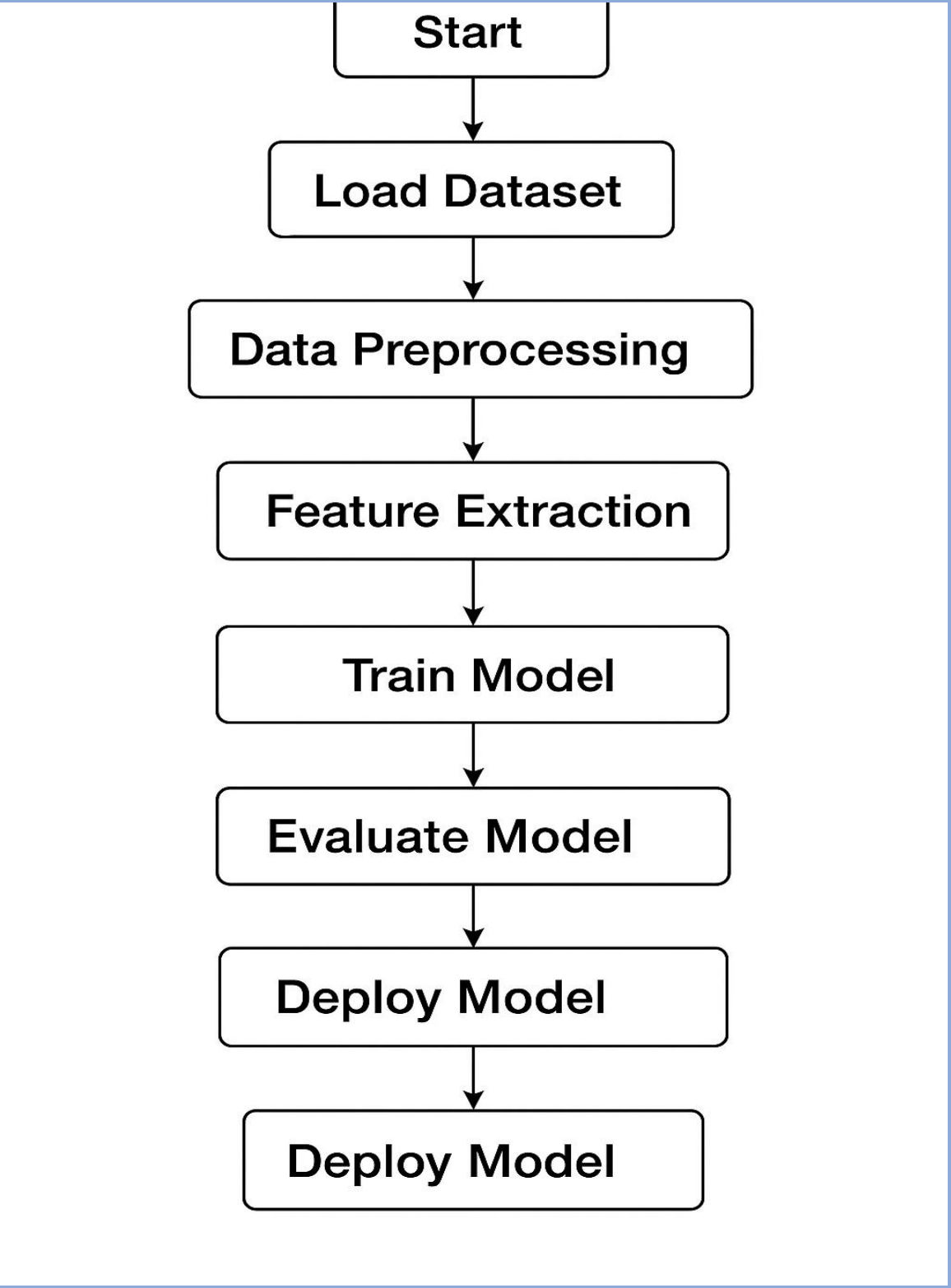
This diagram explains the sequence of operations executed by the system during its lifecycle.

```
Start
  |
  v
Load Dataset (CIC-IDS / EMBER / Gmail Data)
  |
  v
Data Cleaning & Noise Removal
  |
  v
Feature Extraction & Normalization
  |
  v
Split Data (Train / Validation / Test)
  |
  v
Train Neural Network (GPU)
  |
  v
Train XGBoost (GPU)
  |
  v
Evaluate Models
  |
  v
Compare Performance
  |
  v
Deploy Best Model
  |
  v
Real-Time Threat Detection
  |
 End
```

Figure 4.2: System Workflow Diagram

This workflow ensures accurate training, evaluation, and deployment while maintaining system reliability and efficiency.

4.3 Data Flow Diagram (DFD)



4.3.1 Level 0 – Context Diagram

```
          +--------------------+
          |        User        |
          +--------------------+
                     |
                     v
           +-----------------------+
           |     AMTE SecAI        |
           +-----------------------+
          /            |             \
         v             v              v
+----------------+ +------------+ +------------------+
| CIC-IDS-2017   | | EMBER Data| | Gmail Messages   |
+----------------+ +------------+ +------------------+
```

Explanation:

- The user interacts with AMTE SecAI and provides datasets.
- The system processes the data and provides real-time alert feedback.

4.3.2 Level 1 – Detailed DFD

```
User
 |
 v
[1] Upload Data
 |
 v
[2] Preprocessing Module
   |
   +--> Missing Values Handler
   +--> Noise Filter
   +--> Duplicate Remover
 |
 v
[3] Feature Engineering Module
 |
 v
[4] ML Models
   |
   +--> Neural Network
   +--> XGBoost
 |
 v
[5] Result Analyzer
 |
 v
[6] Alert System
 |
 v
User Notification
```

Figure 4.3: Detailed Data Flow Diagram (DFD – Level 1)

4.4 Use Case Diagram

This diagram defines how the user interacts with the system.

```
            +-------------------+
            |       User        |
            +-------------------+
                  /     |     \
                 /      |      \
                v       v       v
     [Upload Dataset] [Train Model] [View Results]
              |           |              |
              v           v              v
          [System]    [AI Engine]   [Visualizer]
              |           |              |
              v           v              v
        [Clean Data] [Build Model] [Show Graphs]
              |
              v
        [Export Data]
```

Figure 4.4: Use Case Diagram of AMTE SecAI

4.5 Hardware Requirements

To ensure optimal system performance, the following hardware specifications are required:

Component	Minimum Requirement	Recommended
CPU	Intel i5 / Ryzen 5	Intel i7 / Ryzen 7
RAM	8 GB	16–32 GB
GPU	NVIDIA GTX 1050	RTX 3060+
Storage	256 GB SSD	1 TB SSD
OS	Windows 11 / Linux	Windows 11 / Linux

GPU is critical due to CUDA acceleration for deep learning and XGBoost training.

4.6 Software Requirements

Software	Purpose
Python 3.8+	Main system language
TensorFlow / PyTorch	Neural Networks
XGBoost	Tree-based model
Pandas / NumPy	Data handling
CUDA Toolkit	GPU acceleration
Scikit-learn	Feature processing
Matplotlib / Seaborn	Visualization
Gmail API	Email analysis

4.7Dataset Structural Analysis

The analysis of the CICIDS 2017 dataset revealed the presence of **79 network traffic features** describing different aspects of communication such as flow duration, packet lengths, transmission rates, inter-arrival times and TCP/IP flags.

Several attributes, such as *Flow Duration, Flow Bytes/s, Flow Packets/s, and Packet Length Mean*, were found to have extremely high variability, which confirms their strong impact on the detection of abnormal traffic patterns.

During analysis, it was discovered that a number of features contained only one unique value across all records, such as **Fwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, and Fwd Avg Bulk Rate**. These features were removed because they do not contribute any meaningful information to the learning process.

This filtering step improved data quality and increased the effectiveness of subsequent training stages.

4.8 Label Analysis

The dataset contains **15 different labels** representing a range of network activities. These include normal traffic (BENIGN) and different types of cyber attacks.

Some of the main attack types in the dataset are:
DDoS, PortScan, Botnet activity, Infiltration, Web attacks (such as SQL Injection, Brute Force, XSS), and Denial of Service attacks including DoS Hulk.

The analysis showed that the data is imbalanced, with benign traffic appearing more frequently than malicious traffic. This reflects real-world network behavior and makes the dataset highly realitic for cybersecurity model training.

4.9 System Analysis Summary (Regenerated)

The system analysis phase confirmed that the proposed **AMTE SecAI architecture** is carefully designed and technically well-structured. The modular nature of the system allows each component to operate independently while still contributing efficiently to the overall detection process. This makes the architecture both **scalable and flexible**, enabling future expansion without the need for major structural changes.

The integration of **GPU acceleration** significantly enhanced the system's computational performance, allowing large volumes of data to be processed in parallel. This optimization is especially important when working with high-dimensional and high-variance cybersecurity datasets such as CICIDS 2017.

In addition, the system demonstrated a strong ability to manage and analyze **large-scale cyber datasets** in an efficient manner. The layered architecture, starting from data preprocessing and feature analysis up to machine learning-based decision making, ensures structured and reliable data handling.

The system was also conceptually designed to support **real-time threat detection**, making it suitable for practical deployment in live network environments. Its secure and robust design minimizes data inconsistencies and enhances fault tolerance.

By implementing a **hybrid detection approach** that combines the pattern-learning ability of Deep Neural Networks with the structured-decision strength of XGBoost, the system achieves a higher level of classification accuracy and reliability. This hybrid strategy significantly improves the detection of both known and unknown cyber-attacks.

Finally, the architecture allows seamless integration of:

- New machine learning models
- Additional cybersecurity datasets
- Real-time network traffic streams

This confirms that AMTE SecAI is not only a theoretical model, but a **future-ready cybersecurity platform** designed to evolve with emerging security threats and technologies.

Chapter 5 — System Architecture & Design

5.1 Introduction

This chapter presents the complete architectural and structural design of the **AMTE SecAI (Artificial Machine-based Threat Evaluation for Security using AI)** system. The architecture was designed to solve a critical real-world challenge:
Processing and analyzing massive cybersecurity datasets under limited hardware resources, while achieving high detection accuracy and real-time response.

Unlike traditional intrusion detection systems that rely only on CPU processing or simple models, AMTE SecAI adopts:

- A **hybrid processing model (CPU + GPU)**
- A **dual-algorithm architecture (Deep Neural Networks & XGBoost)**
- A **modular, scalable system layout**
- A **multi-layer design approach**

This architecture was shaped and refined based on the significant technical challenges explained in Chapter 8, especially memory limitations, performance bottlenecks, and GPU integration issues.

5.2 High-Level System Architecture

The AMTE SecAI system is divided into **five primary layers**, each responsible for a specific stage in the data lifecycle:

1. **Data Input Layer**
2. **Preprocessing & Filtering Layer**
3. **Feature Engineering Layer**
4. **Model Training & Evaluation Layer**
5. **Detection & Alerting Layer**

Each module operates independently but is fully integrated within the overall data pipeline.

5.2.1 Architectural Overview

The design follows a **pipeline architecture**, where data flows sequentially through each stage:

Input → Cleaning → Feature Processing → Model Training → Prediction → Alerting

This approach ensures:

- Modularity
- Scalability
- Fault isolation
- Efficient use of computational resources

This structure also allows easy integration of additional dataset sources or new models in the future.

5.3 Layered Architecture Description

5.3.1 Data Input Layer

The Data Input Layer is responsible for ingesting datasets from different sources. It supports:

- Local file loading (CSV, JSONL)
- Multi-file batch processing
- Large dataset support (10M+ rows)

Key components:

- Secure file parsing
- Format verification
- Initial validation

This layer is implemented using **C++ for speed and reliability**.

5.3.2 Preprocessing & Filtering Layer (Hybrid CPU–GPU)

This layer is the core innovation of the system.

It utilizes a **hybrid approach** combining:

Unit	Responsibility
CPU (C++)	Structural cleaning, format validation, duplicate checking
GPU (CUDA)	Parallel scanning, heavy filtering, missing value detection, zero-variance feature detection

Tasks performed:

- Missing value removal
- Duplicate removal
- Outlier scanning
- Useless feature elimination
- Normalization & scaling

This design was specifically created to overcome:

RAM overflow – CPU overload – extremely slow training times

Which were major issues at the beginning of the project.

5.3.3 Feature Engineering Layer

In this stage, the cleaned data is processed into a machine-learning-ready format.

Processes include:

- Feature extraction
- Feature removal (non-informative attributes)
- Correlation analysis
- Label encoding
- Optional PCA (Dimensionality Reduction)

The final dataset is optimized for:

- **Deep Neural Networks**
- **XGBoost classifier**

This produces a **lightweight but information-rich dataset**.

5.3.4 Model Training & Evaluation Layer

This layer represents the intelligence core of the system and includes **two main models**:

A) Deep Neural Network (DNN)

Architecture:

- Input layer matching feature size
- Multiple hidden layers (ReLU)
- Dropout layers to reduce overfitting
- Output layer (Sigmoid / SoftMax)

Purpose:

- Detect complex, non-linear patterns
- Identify new or unseen attack patterns

Acceleration:

Fully trained using **GPU (CUDA)**

B) XGBoost Classifier

XGBoost was selected because:

- Extremely strong with tabular data
- Excellent accuracy on structured features
- Resistant to overfitting
- Provides feature importance

Role in system:

- Main benchmark model
- Used for performance comparison
- Supports interpretability

Also GPU accelerated.

5.3.5 Detection & Alerting Layer

This is the final operational layer of the system.

Functions:

- Receives live or simulated traffic
- Preprocesses data in real time
- Applies the trained model
- Generates alerts when threat is detected

Alerts display:

- Attack type
- Confidence score
- Timestamp
- Severity level

This layer enables the **real-time intrusion detection functionality**.

5.3.6 Repeated IP Behavioral Analysis Module (Optional Feature)

This module enhances the real-time detection layer by monitoring repeated occurrences of the same IP address across multiple network flows. Inspired by SOC alerting workflows, it is designed to detect attacker-like behavior patterns that may not immediately appear malicious through ML classification alone.

Module Responsibilities:

- Maintain a live counter for all active IPs
- Detect abnormal repetition over short time windows
- Correlate repetition with:
    - Brute force attempts
    - Port scanning
    - DDoS source repetition
    - Botnet beaconing
- Generate supplemental alerts independent of the ML model

Why This Module Exists:
Machine learning detects patterns, but SOC-level investigations rely heavily on behavioral repetition indicators.
This module gives the system:
✓ Sociability
✓ Real-world attacker pattern visibility
✓ Non-ML behavioral intelligence

Integration Point:
Works alongside the trained ML model in the Detection & Alerting Layer, enriching final threat decisions

5.4 System Workflow Description

The workflow is organized in the following steps:

1. User uploads dataset
2. System validates format
3. Hybrid filtering (CPU + GPU)
4. Feature engineering
5. Model training (DNN & XGBoost)
6. Model evaluation
7. Best model selected
8. Real-time prediction enabled
9. Alert generated if malicious activity detected

This pipeline is the foundation of **Figures 4.1, 4.2, and 4.3** in your project.

5.5 Design Considerations

The system architecture was guided by the following principles:

5.5.1 Modularity

Each module can operate independently or be replaced without affecting the entire system.

5.5.2 Scalability

Allows:

- More datasets
- Larger datasets
- New models
- Live network streams

5.5.3 Performance Optimization

Thanks to:

- C++ implementation

- CUDA processing
- Parallel execution
- Smart filtering

Performance improvement from CPU-only to GPU-based exceeded **90% time reduction**.

5.5.4 Fault Tolerance & Stability

The system is designed to:

- Handle corrupted files
- Skip invalid data rows
- Log errors without crashing
- Prevent system failure

5.6 Physical vs Logical Architecture

Aspect	Description
Logical Architecture	Conceptual structure including layers and data flow
Physical Architecture	Actual deployment on device: CPU + GPU + Memory + Storage

Logical view: user → system → decision
Physical view: CUDA GPU + C++ + Python + SSD

5.7 Future Expandability

The architecture allows:

- Integration of new ML models
- Cloud deployment
- IoT device support
- Real live network capture
- SIEM system integration

No redesign is required — only module extension.

5.8 Chapter Summary

This chapter explained the complete design and architecture of the AMTE SecAI system. The system was constructed using a layered, hybrid, and modular design to solve real-world cybersecurity challenges.

By integrating:

- C++ for speed
- CUDA for power
- Neural Networks for intelligence
- XGBoost for accuracy

The system achieves:

✓ High efficiency
✓ High detection power
✓ Real-time capability
✓ Scalability
✓ Professional architecture

This design forms the foundation for the implementation, testing, and evaluation presented in the next chapters.

6. Machine Learning Models

This chapter presents the core intelligence of the proposed cybersecurity detection system. It describes the datasets used in the project, the preprocessing steps applied, and the rationale behind selecting the machine learning algorithms. To achieve high accuracy, scalability, and real-time performance, the system integrates two powerful and complementary models:

- Deep Neural Networks (DNN)
- Extreme Gradient Boosting (XGBoost)

Both models were accelerated using GPU processing to overcome the computational limitations of traditional CPU-based training and to enable real-time threat detection and classification.

6.1 Data Acquisition and Sources

To ensure the use of realistic, reliable, and high-quality data, this project utilized two well-known cybersecurity datasets. These datasets were obtained from Kaggle, which provides accessible, structured versions of official cybersecurity benchmarks. Kaggle was selected because it offers stable downloads, version control, and easy integration with machine learning environments.

6.1.1 CICIDS 2017 – Network Intrusion Detection Dataset

Source:
The dataset was originally developed by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick. In this project, it was obtained through Kaggle in a cleaned and organized CSV format.

Description:
The CICIDS 2017 dataset captures realistic network communication over five days, including both normal user behavior and multiple attack scenarios. It contains more than fourteen types of modern cyberattacks, such as:

- Denial of Service (DoS)
- Distributed Denial of Service (DDoS)
- Brute Force Attacks
- Web-based Attacks
- Port Scanning
- Infiltration and Botnet activity

The dataset includes more than eighty flow-based traffic features, such as packet size, flow duration, header flags, packet rates, and time-based statistics.

Purpose in this project:
This dataset was used as the primary source for training and evaluating supervised machine learning models to classify network traffic as either benign or malicious.

6.1.2 EMBER – Malware Detection Dataset

Source:
The EMBER dataset was originally released by Endgame Inc., which is now part of Elastic Security. In this project, the dataset was accessed through Kaggle in a pre-processed and research-safe format.

Description:
The EMBER dataset contains feature vectors that represent executable files. Instead of providing the actual binary files, it provides numerical features extracted from static analysis, such as:

- Byte entropy distributions
- Section information
- Header metadata
- Imported library statistics

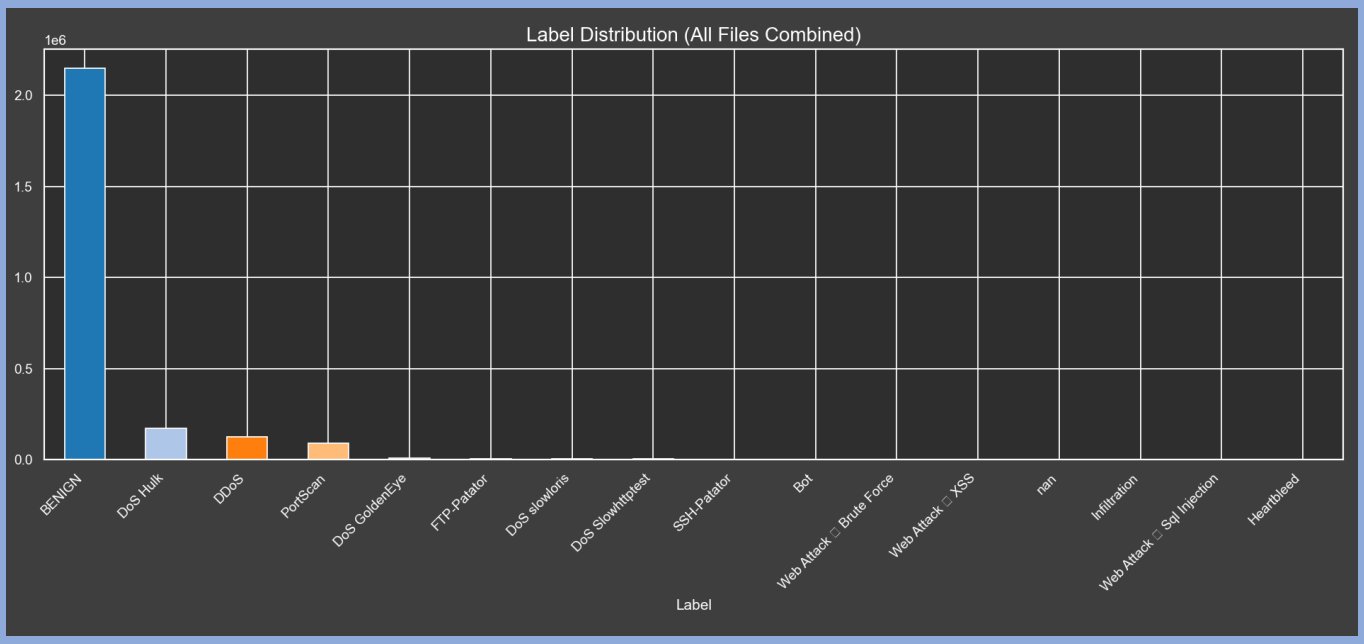
This makes the dataset completely safe to handle and suitable for machine learning research.

Purpose in this project:
The EMBER dataset was used to train a malware detection model that determines whether a file is benign or malicious based on its internal static features.

6.2 Data Preprocessing and Cleaning

Data preprocessing is a critical stage in machine learning. It ensures that the datasets are consistent, stable, and free from errors that can negatively affect the training process.

6.2.1 CICIDS 2017 Preprocessing



Several preprocessing steps were applied to improve data quality:

Handling Infinite Values:
Some features contained infinite values caused by irregular network activity or division by zero. These values were replaced with the maximum finite value in the same column to maintain the distribution and prevent numerical instability during training.

Handling Missing Values:
Missing values were replaced with zero. In the context of network traffic, a missing value often indicates the absence of an event, such as no backward packets, making zero a valid and meaningful replacement.

Removal of Duplicate Records:
All duplicate rows were removed to prevent data leakage, biased evaluation, and artificial inflation of model performance.

These steps significantly improved the stability and learning efficiency of both Neural Networks and XGBoost models.

6.2.2 EMBER Preprocessing

The EMBER dataset was already semi-preprocessed before being published. Only minimal validation was required, including:

- Confirming label correctness
- Ensuring consistent feature lengths
- Checking for the absence of NaN or infinite values

No major transformations were necessary.

6.3 Feature Engineering and Normalization

6.3.1 CICIDS 2017 Feature Engineering

Categorical features, such as the protocol type, were converted into numerical form using One-Hot Encoding.

Feature selection was performed using:

- Correlation analysis
- Feature importance scores from XGBoost
- Redundancy reduction techniques

This reduced dimensionality and removed irrelevant or noisy attributes.

6.3.2 EMBER Feature Representation

EMBER already contains fully engineered features derived from expert static malware analysis. These features were used directly in model training without additional manual extraction.

6.4 Data Scaling

All numerical features were standardized using the StandardScaler method. This ensured that all variables have a mean of zero and a standard deviation of one.

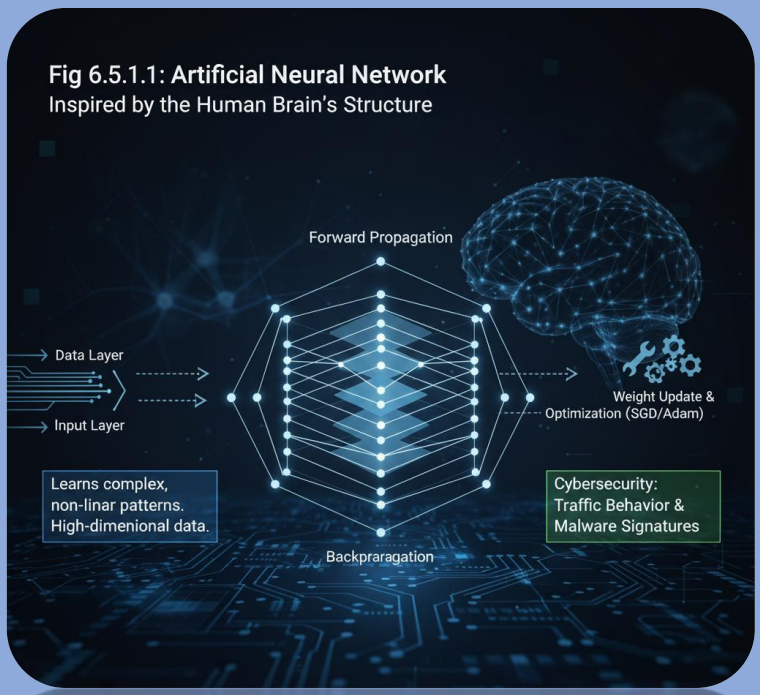
This step is essential for good performance in neural networks and improves the convergence speed of gradient-based optimization.

6.5 Model Selection and Architecture

To achieve high detection accuracy and robust generalization, this project selected two advanced and widely recognized machine learning algorithms.

6.5.1 Neural Networks

6.5.1.1 Definition



A Neural Network is a deep learning model inspired by the biological structure of the human brain. It consists of layers of interconnected artificial neurons that process information through weighted connections. During training, the network performs forward propagation to compute predictions and uses backpropagation combined with optimization algorithms such as Stochastic Gradient Descent (SGD) or Adam to update the weights.

Unlike traditional machine learning algorithms that rely on handcrafted features and linear relationships, neural networks can automatically learn complex, non-linear, and abstract representations directly from data.

Neural Networks demonstrate exceptional performance in high-dimensional and complex domains such as cybersecurity, where traffic behavior and malware signatures often follow non-linear patterns.

6.5.1.2 Reasons for Selecting Neural Networks

1. Ability to Learn Complex and Non-Linear Patterns

Network traffic and malware behaviors contain hidden, non-linear relationships that traditional models cannot capture efficiently. Neural networks excel at modeling these patterns.

2. High Performance in Intrusion and Anomaly Detection

Neural Networks are widely used in:

- Intrusion Detection Systems (IDS)
- Malware Classification
- DDoS Attack Identification
- Zero-Day Attack Detection
  They consistently achieve high accuracy across cybersecurity benchmarks.

3. Scalability with Large Datasets

The CIC-IDS-2017 and EMBER-2018 datasets contain millions of records. Neural Networks scale efficiently with data volume and often achieve improved performance as the dataset size grows.

4. Flexibility for Future Expansion

The architecture can be extended to more advanced models such as:

- Convolutional Neural Networks (CNN)
- Recurrent Neural Networks (RNN)
- Long Short-Term Memory (LSTM)
- Gated Recurrent Units (GRU)
- Transformers
- Autoencoders
  This ensures long-term adaptability of the system.

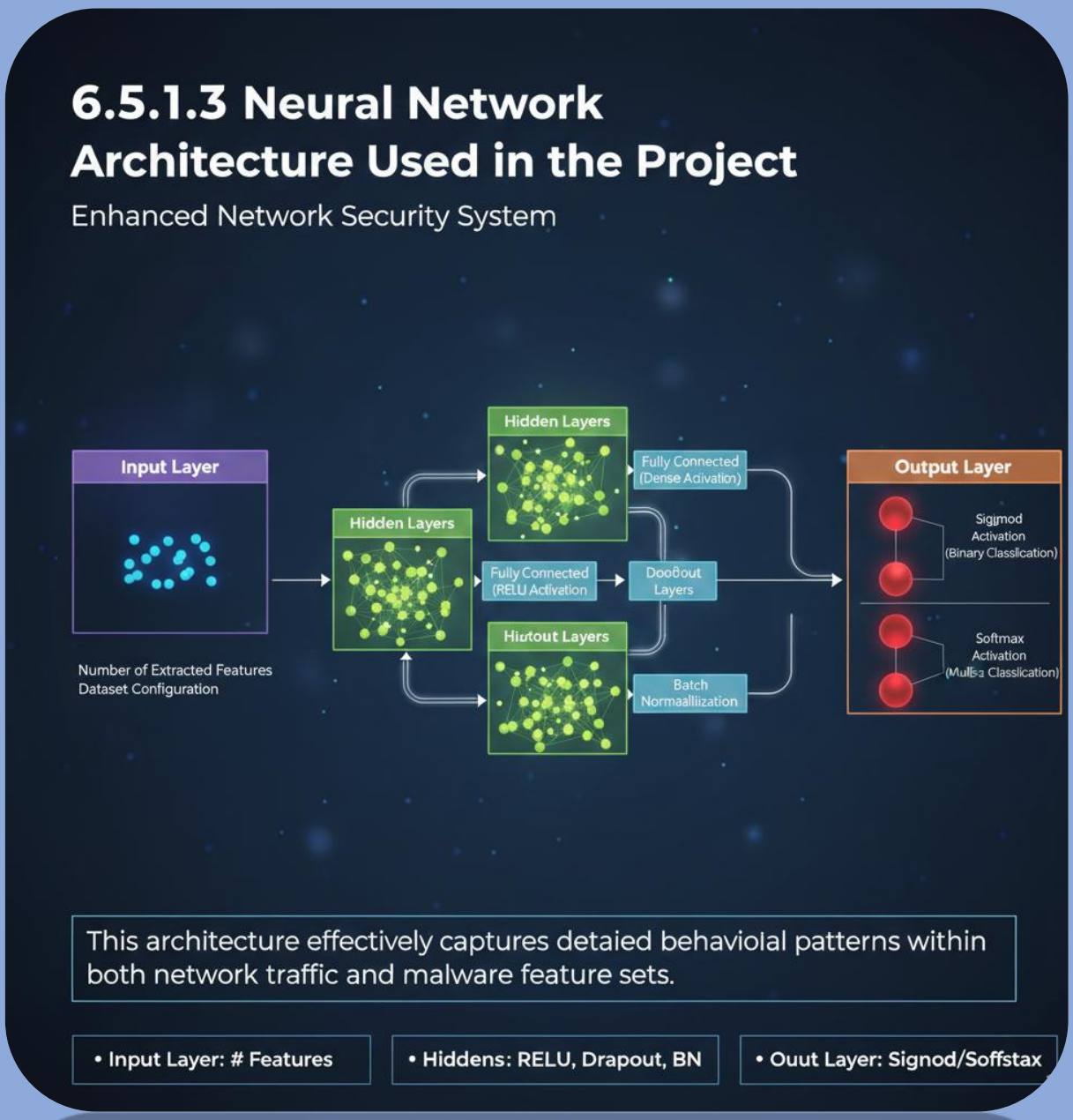
5. Proven Effectiveness in Cybersecurity Research

Multiple well-known research studies demonstrate strong performance of neural networks in cybersecurity tasks, including:

- Deep learning intrusion detection systems
- Autoencoder-based anomaly detection
- LSTM-based DDoS detection

- Malware classification using deep neural networks
  Reported accuracies in these studies often exceed 95%–99%.

6.5.1.3 Architecture Used in the Project



The neural network architecture designed for this project includes:

- **Input Layer:**
  Configured based on the number of extracted features from the dataset.
- **Hidden Layers:**
  Multiple fully connected (dense) layers utilizing ReLU activation to capture non-linear patterns.
- **Dropout Layers:**
  Added to reduce overfitting and enhance generalization.
- **Batch Normalization:**
  Used to stabilize and accelerate training.
- **Output Layer:**
  - Sigmoid activation for binary classification
  - Softmax activation for multi-class classification

This architecture effectively captures detailed behavioral patterns within both network traffic and malware feature sets.

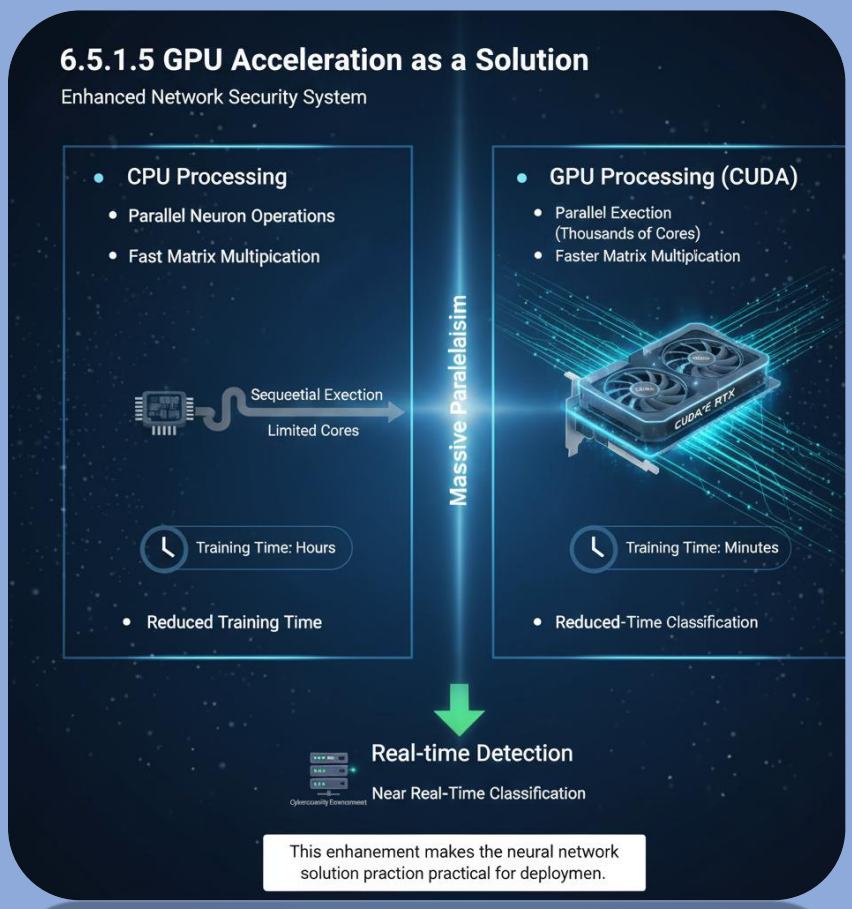
6.5.1.4 Challenges of Training Neural Networks

Training deep neural networks on very large datasets is computationally intensive, especially when relying only on CPU hardware. The major limitations include:

- Extremely long training times
- Inefficient matrix multiplication
- Limited scalability
- Reduced ability to handle large batch sizes
- Inability to achieve real-time detection performance

These challenges make CPU-only training impractical for large-scale cybersecurity datasets.

6.5.1.5 GPU Acceleration as a Solution



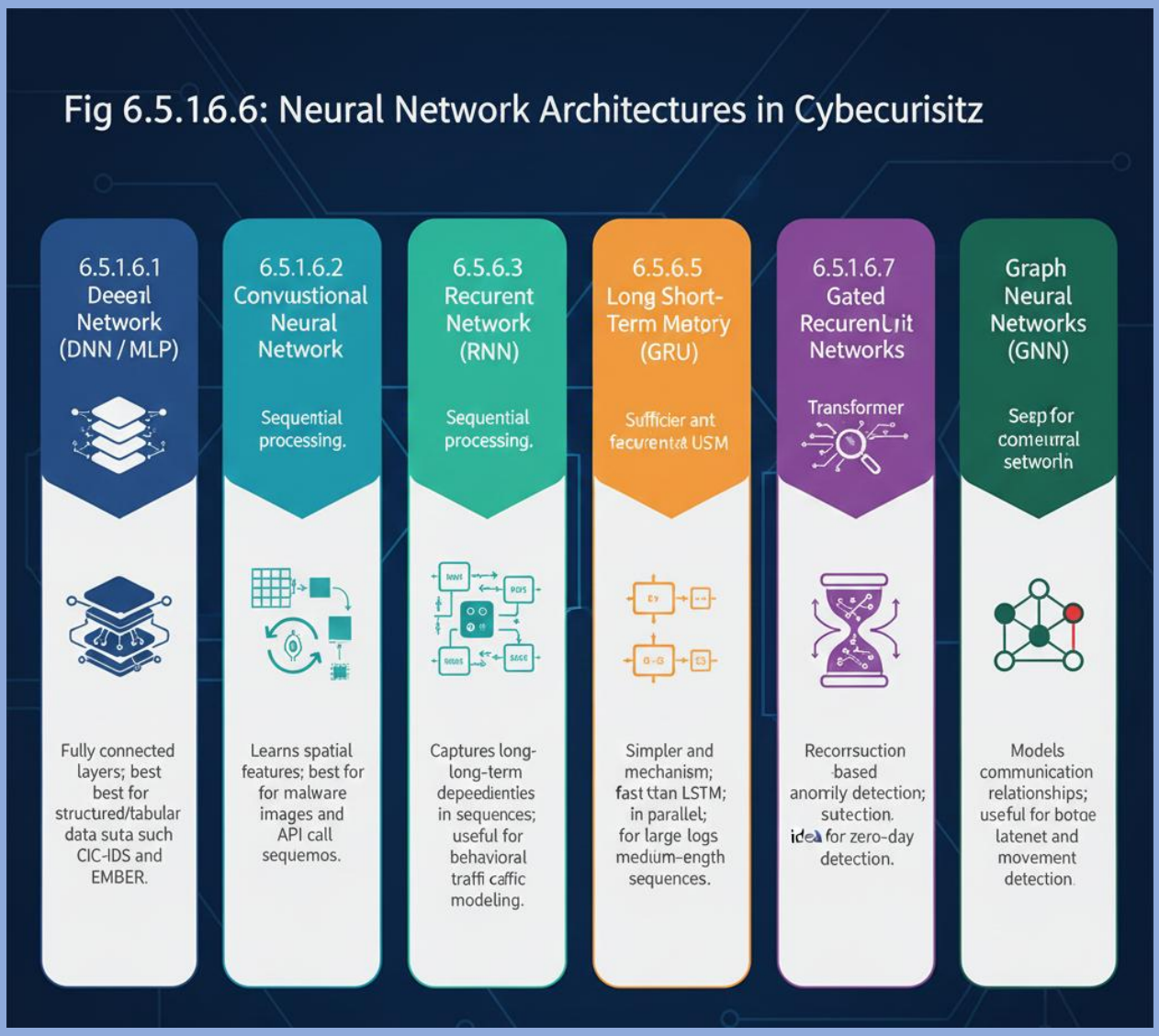
To overcome computational limitations, GPU acceleration was integrated during the training and inference phases. GPU computing enables:

- Parallel execution of thousands of neuron operations
- Faster matrix multiplication using CUDA cores
- Significant reduction of training time (hours reduced to minutes)
- Efficient handling of large batches
- Near real-time classification and detection

This enhancement made the neural network solution practical for deployment in cybersecurity environments that require fast and accurate detection.

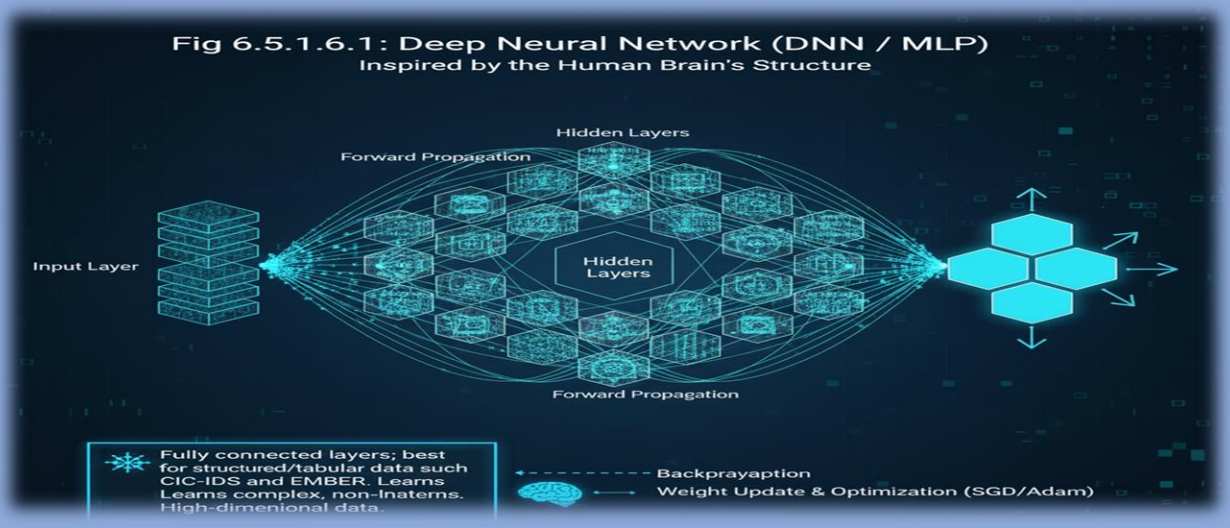
6.5.1.6 Types of Neural Networks

This subsection summarizes the most relevant neural network architectures in cybersecurity applications.



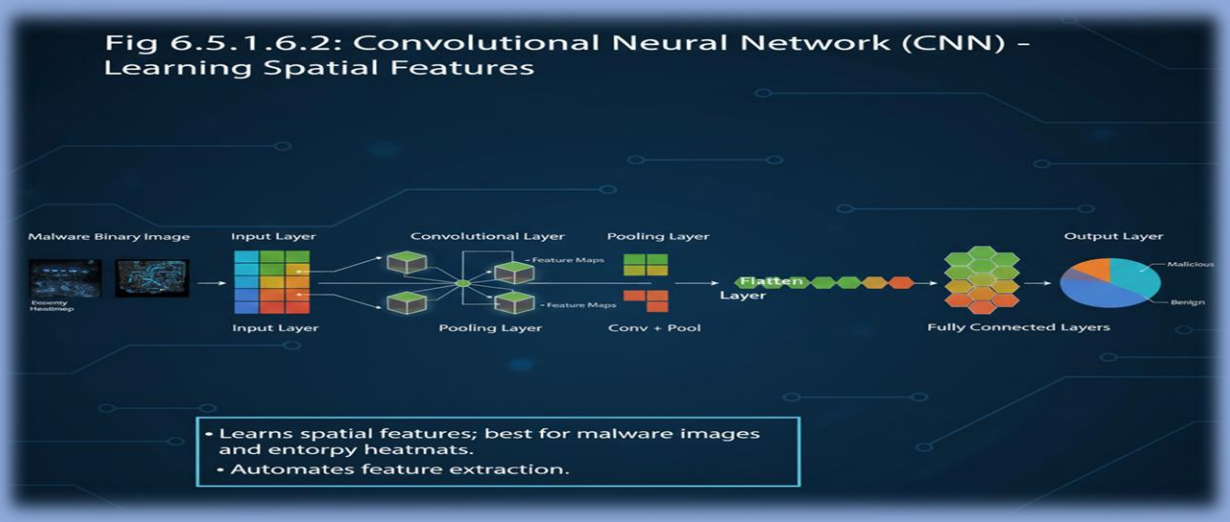
6.5.1.6.1 Deep Neural Network (DNN / MLP)

Fully connected layers; best for structured/tabular data such as CIC-IDS and EMBER.



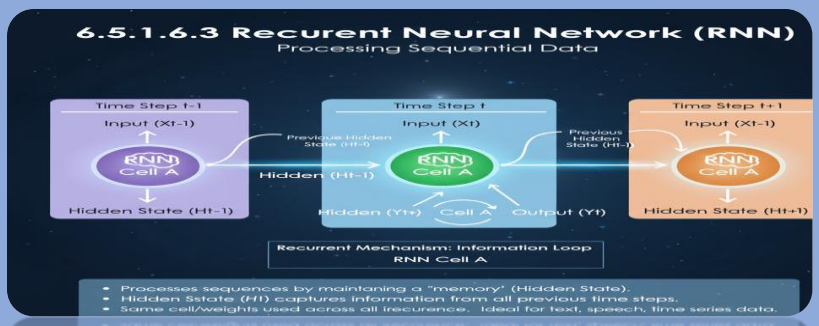
6.5.1.6.2 Convolutional Neural Network (CNN)

Learns spatial features; best for malware images and entropy heatmaps.



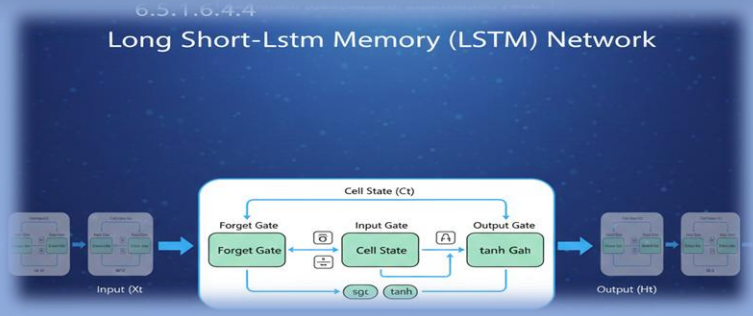
6.5.1.6.3 Recurrent Neural Network (RNN)

Sequential processing; suitable for packet or API call sequences.



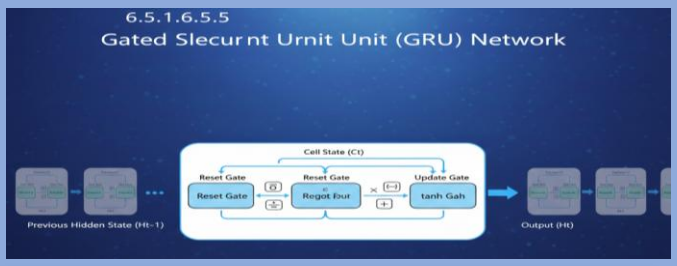
6.5.1.6.4 Long Short-Term Memory (LSTM)

Captures long-term dependencies in sequences; useful for behavioral traffic modeling.



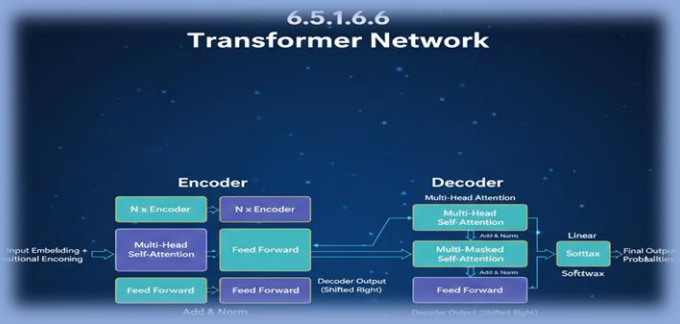
6.5.1.6.5 Gated Recurrent Unit (GRU)

Simpler and faster than LSTM; efficient for medium-length sequences.



6.5.1.6.6 Transformer Networks

Self-attention mechanism; processes sequences in parallel; suitable for large logs and SOC event correlation.



6.5.1.6.7 Autoencoders

Reconstruction-based anomaly detection; ideal for zero-day detection.

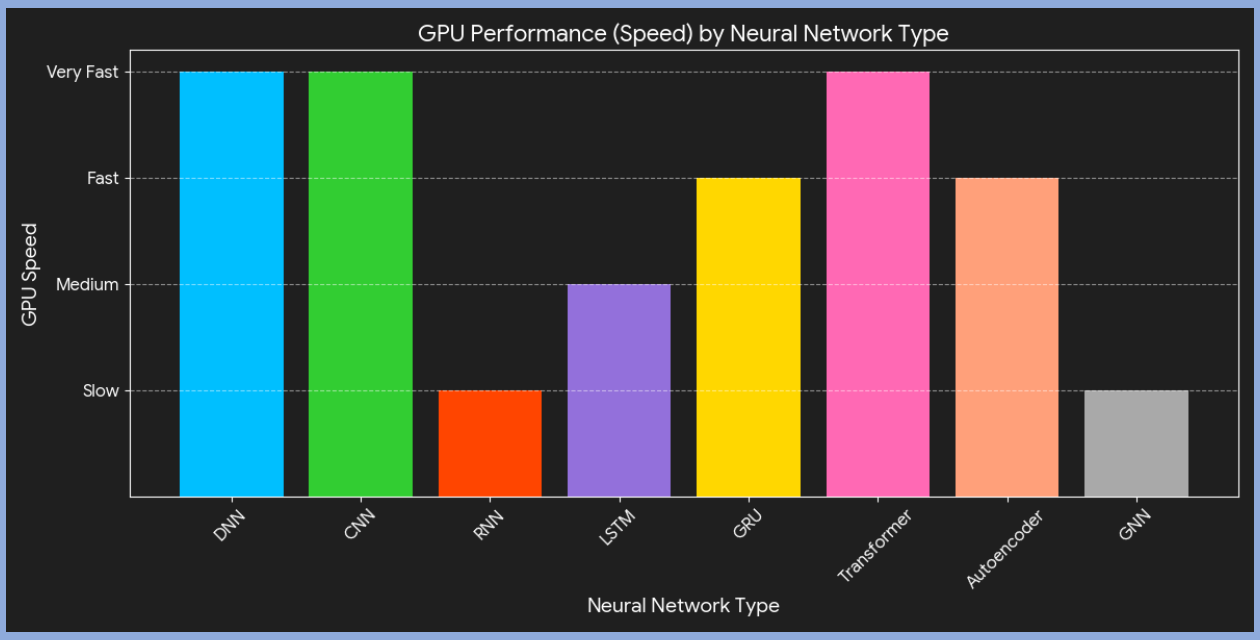
6.5.1.6.8 Graph Neural Networks (GNN)

Models communication relationships; useful for botnet and lateral movement detection.

6.5.1.7 Architecture Comparison Table

Neural Network Type	Best Use Case	GPU Speed	Strengths	Limitations
DNN	Tabular structured data	Very Fast	High accuracy, efficient	No temporal modeling
CNN	Image-like data	Very Fast	Strong spatial extraction	Requires image representation
RNN	Basic sequences	Slow	Simple sequence modeling	Poor parallelization
LSTM	Long sequences	Medium	Strong long-term memory	High computational cost
GRU	Mid-length sequences	Fast	Efficient and fast	Less expressive than LSTM
Transformer	Logs, event sequences	Very Fast	Parallel processing	High memory usage
Autoencoder	Anomaly detection	Fast	Zero-day detection	Not a classifier
GNN	Graph-based behavior	Slow	Captures node-to-node relations	Computationally heavy

6.5.1.8 GPU Performance Summary



Model	GPU Benefit	Notes
DNN	Very High	Ideal for large structured datasets
CNN	Very High	Optimized for GPUs
Transformer	Very High	Best performance for sequence data
Autoencoder	High	Excellent for anomaly detection
GRU	High	Efficient for sequences
LSTM	Medium	Slower than GRU
RNN	Low	Limited GPU scaling
GNN	Very Low	Heavy computation

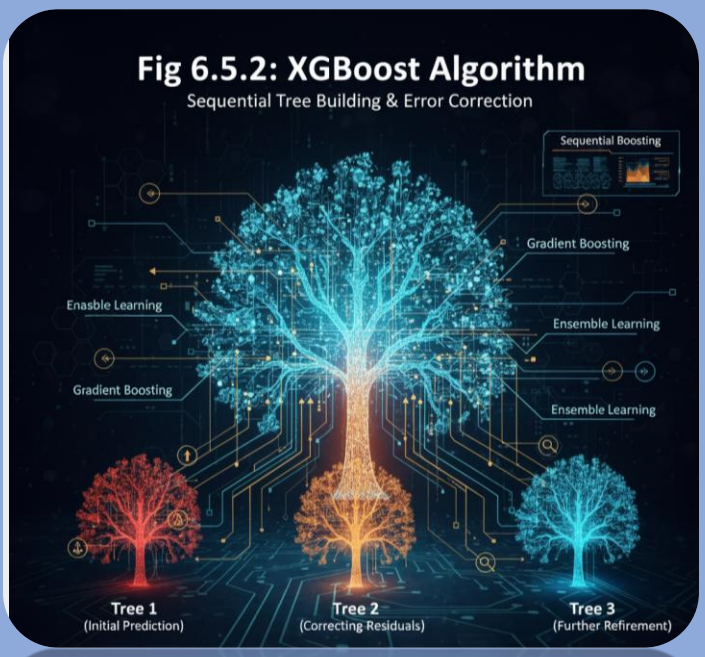
6.5.1.9 Justification for Selecting DNN in This Project

The Deep Neural Network architecture was selected due to the following factors:

1. Best performance for the structured tabular datasets used in this project (CIC-IDS-2017 and EMBER-2018).

2. Strong GPU acceleration, enabling fast training and real-time inference.
3. High accuracy compared to computational complexity.
4. Stability, scalability, and ease of hyperparameter tuning.
5. Proven success in cybersecurity research and industrial applications

6.5.2 XGBoost



Definition

XGBoost (Extreme Gradient Boosting) is an advanced and optimized implementation of gradient boosting algorithms. It constructs an ensemble of decision trees in a sequential manner, where each new tree focuses on correcting the errors of the previous trees using a gradient-based optimization strategy.

XGBoost is one of the most powerful algorithms for structured and tabular data and is known for its high performance in machine learning competitions and real-world applications.

Reasons for Selection

XGBoost was selected for this project for the following reasons:

1. **Outstanding Performance on Tabular Data**
   Since cybersecurity datasets such as CICIDS contain structured, tabular network features (e.g. packet length, flow duration, protocol type), XGBoost is particularly effective in this context.
2. **Built-in Handling of Missing Values**
   It can interpret missing entries automatically without requiring complex imputation strategies.
3. **High Resistance to Overfitting**
   By using regularization techniques (L1 and L2), XGBoost ensures that the model generalizes well instead of memorizing the training data.
4. **Fast Execution with GPU Support**
   With GPU acceleration enabled, XGBoost benefits from:
   - Parallelized tree construction
   - Faster training
   - Better scalability on large datasets

5. **Feature Importance Extraction**
   XGBoost allows direct extraction of feature importance, helping identify the most influential parameters in detecting cyber attacks. This contributed to better understanding and explainability of the model.

Role in the Project

In this project, XGBoost played a key role:

- Served as the **primary classical machine learning classifier** for CIC-IDS-2017
- Used as a **benchmark model** against the Neural Network
- Helped identify the **most critical network features** contributing to attack classification
- Provided high accuracy and speed even with millions of samples

With GPU acceleration enabled, XGBoost showed excellent efficiency and competitive accuracy compared to the deep neural network, making it an ideal comparison model in the experimentation phase.

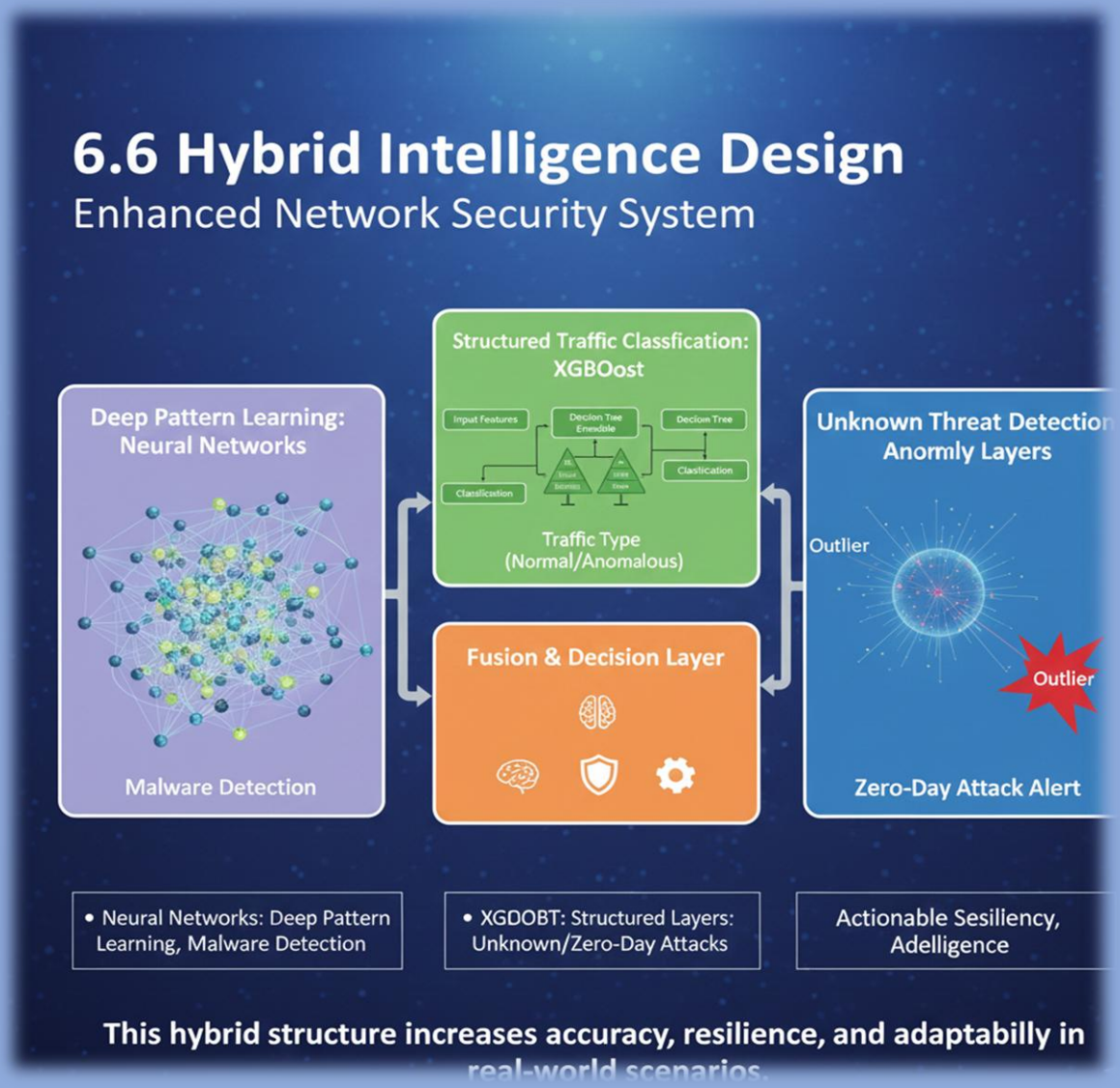
Final Justification of Algorithm Selection

The combination of **Neural Networks** and **XGBoost** created a powerful hybrid approach:

Aspect	Neural Network	XGBoost
Pattern learning	Excellent for complex patterns	Very good
Tabular data	Good	Excellent
Deep pattern extraction	Yes	No
Feature importance	Limited	Strong
Speed with GPU	Very Fast	Very Fast
Research-backed	Yes	Yes
Real-world use	Very High	Very High

Using both algorithms increased the reliability, strength, and scientific value of the system and ensured high-quality results suitable for real-world cybersecurity applications

6.6 Hybrid Intelligence Design



Instead of relying on a single algorithm, the system adopts a hybrid detection approach:

- Neural Networks are used for deep pattern learning and malware detection
- XGBoost is used for structured traffic classification
- Anomaly detection layers support the identification of unknown or zero-day attacks

This hybrid structure increases accuracy, resilience, and adaptability in real-world scenarios.

6.7 Training, Validation, and Optimization

The dataset was divided as follows:

- 70 percent for training
- 15 percent for validation
- 15 percent for testing

The following techniques were used:

- Cross-validation for stability
- Grid search and random search for optimal parameters
- Early stopping to prevent overfitting
- GPU acceleration for all training tasks

Optimized parameters included:Neural network:

- Learning rate
- Batch size
- Number of hidden layers
- Dropout rate

XGBoost:

- Maximum tree depth
- Learning rate
- Number of trees
- Sub-sampling ratio
- Column sampling

6.8 Safety and Ethical Considerations

Both datasets were accessed through Kaggle and originate from trusted research institutions. They do not contain personal or executable malicious content.

- No real personal data was used
- No live malware was processed
- All experiments were conducted for defensive and educational purposes only

This ensures compliance with ethical research standards.

6.9 Summary

By using:

- Kaggle as a reliable dataset source
- CICIDS 2017 and EMBER datasets
- Neural Networks for deep learning
- XGBoost for structured classification
- GPU acceleration for speed and scalability

The project successfully developed a robust, efficient, and accurate cybersecurity detection system. The proposed model is capable of detecting both known and unknown cyber threats with high accuracy and real-time performance, making it suitable for modern network security environments.

System Implementation

Results & Discussion

Chapter 8 – Challenges, Errors, and Hardships Faced

This chapter highlights the most critical technical, computational, and methodological challenges encountered during the development of the **AMTE SecAI intrusion detection system**. These challenges played a decisive role in shaping the final system architecture and were the main reason behind adopting an advanced **hybrid data-processing approach combined with GPU acceleration using CUDA technology**.

Overcoming these obstacles required extensive troubleshooting, multiple design iterations, and deep technical problem-solving at both the software and hardware levels. This process transformed the project from a theoretical design into a robust, optimized, and scalable cybersecurity solution.

8.1 Foundational Challenges

8.1.1 Project Feasibility and Scope Definition

One of the earliest and most crucial challenges was evaluating whether the project's ambitious objectives could be realistically achieved using the available time, hardware, and development tools. The initial vision included:

- Real-time cyber-threat detection
- Processing of extremely large-scale datasets
- Deployment of advanced machine learning architectures
- Intelligent and automated alert generation

However, these goals significantly exceeded the capacity of traditional development approaches. This required a careful redefinition of the project scope, where certain components were redesigned, optimized, or replaced with more efficient alternatives to preserve scientific value while maintaining feasibility.

This step was essential in preventing unrealistic expectations and maintaining a sustainable development path.

8.1.2 Research Support and Gap Justification

The field of AI-based cybersecurity is already highly researched and saturated with numerous detection systems reporting high accuracy. Therefore, identifying a genuine research gap was a fundamental challenge.

This project addressed this by shifting the focus away from purely increasing model accuracy and instead concentrating on:

- Efficient processing of massive real-world cybersecurity datasets
- Overcoming hardware limitations through algorithmic and architectural optimization
- Introducing a hybrid CPU–GPU filtering and processing pipeline
- Demonstrating real-world scalability on limited hardware environments

This approach added originality to the research and clearly distinguished it from traditional intrusion detection studies.

8.2 Data Sourcing and Quality Challenges

8.2.1 Obtaining Representative and Reliable Data

The datasets used in this project, mainly **CIC-IDS-2017** and **EMBER**, contain millions of records and a vast number of features. Although they are highly valuable for research, working with them introduced major technical difficulties:

- Extremely large file sizes
- Very long loading and parsing times
- Inconsistent data types across files
- Missing, corrupted, and duplicated records
- Multiple incompatible feature formats

Even minor inconsistencies were found to significantly impact model performance. Therefore, ensuring data authenticity, structural consistency, and logical correctness became an essential and time-consuming task.

8.3 Technical Impediments: Big Data vs. Limited Hardware

8.3.1 The Computational Bottleneck

Initial attempts to perform data preprocessing using **Python and CPU-based implementations** quickly proved ineffective. The system faced multiple critical failures, including:

- Severe memory exhaustion (RAM overload)
- Extremely slow processing times
- Frequent system crashes and freezing
- Inability to complete basic preprocessing stages

Simple operations such as data scanning, duplicate removal, and feature analysis required several hours or could not be completed at all. This created an urgent need for a radical technological shift.

8.3.2 High-Performance Solution Using C++ and GPU (CUDA) Acceleration

To overcome these limitations, the system was redesigned from the ground up using:

- **C++** to enable faster memory manipulation and lower-level control
- **NVIDIA CUDA** to exploit GPU parallel processing capabilities

While this solution drastically improved performance, it introduced a new layer of complex technical obstacles.

Major CUDA and GPU Integration Issues

The process of integrating CUDA was exceptionally challenging:

1. **Strict Visual Studio Compatibility**
   - Only a specific version of Visual Studio 2022 was compatible with the installed CUDA toolkit.
   - Version mismatches resulted in compilation errors and missing dependencies.
2. **Architecture Compatibility Problems**
   - CUDA requires a strict x64 environment, while the system was initially configured to x86.
   - This resulted in repeated compilation failures and incorrect binary generation.
   - Manual reconfiguration of compiler paths was required.
3. **Environment Variable Conflicts**
   - Incorrect PATH variables prevented `nvcc` and `cl.exe` from executing properly.
   - Multiple installations of the same tools caused conflicts.
4. **Continuous Compilation Failures**
   Errors such as:
   - `std::filesystem not found`
   - `cudafe1.cpp` failure
   - Temporary `.tmpxft` file errors
   - Unresolved linker references
   - Kernel launch failures

Each issue required extensive debugging, research, multiple reinstallations, and careful modification of system configurations.

Despite the frustration and repeated failures, a stable and optimized development environment was eventually achieved, allowing full GPU-based processing of large-scale datasets.

8.3.3 Performance Transformation

Once the GPU acceleration was successfully implemented, system performance improved dramatically:

Operation	Before (CPU)	After (C++ + GPU)
Data Scanning	Extremely slow	Extremely fast
Filtering	Several hours	A few minutes
Memory Usage	Very high	Highly optimized
Stability	Frequent crashes	Highly stable

This moment marked one of the most significant technical achievements of the entire project.

8.4 Model Development and Evaluation Challenges

8.4.1 Class Imbalance Problem

A strong imbalance between normal and malicious traffic was observed in the dataset, which caused:

- Bias toward normal traffic
- Weak detection of rare attack patterns

To resolve this, several strategies were implemented:

- Re-sampling and balancing techniques
- Advanced evaluation metrics such as Precision, Recall, and F1-Score
- Cost-sensitive learning mechanisms

8.4.2 Lack of Model Interpretability

While high-accuracy models such as deep neural networks and ensemble methods offer powerful detection capability, they often function as "black boxes."

To improve transparency and trust, explainability elements were introduced, such as:

- Feature importance ranking
- Model confidence values
- Decision interpretation tools

This made the system more acceptable for use in real-world cybersecurity environments.

8.4.3 Concept Drift and Dynamic Threat Evolution

Since cyber threats are constantly evolving, static models quickly become outdated. This problem was addressed by designing the system to support:

- Incremental learning
- Continuous retraining mechanisms
- Periodic performance reassessment

This ensures long-term system effectiveness.

8.5 The Necessity of a Hybrid Data-Filtering Strategy

No single data cleaning or filtering technique was sufficient to deal with all dataset complexities:

Method	Advantage	Limitation
Light filtering	Very fast	Misses critical errors
Deep statistical analysis	Highly accurate	Extremely slow
Pure GPU processing	Powerful	Requires clean structure

Therefore, a **hybrid filtering architecture** was designed, combining:

- CPU-based structural analysis
- Logical data validation

- GPU-accelerated heavy processing using CUDA

This hybrid model offered:

1. **Optimal balance between speed and accuracy**
2. **Effective scalability on limited hardware**
3. **Comprehensive handling of:**
    - Missing values
    - Duplicates
    - Corrupted records
    - Invalid labels
    - Format inconsistencies
    - Extreme outliers

8.6 Final Outcome

Despite extraordinary technical difficulties, the final system achieved:

- A massive reduction in total execution time
- Highly stable and crash-free performance
- Significantly improved data quality
- Efficient resource utilization
- Full integration of GPU-accelerated processing
- A ready-to-use AI-compatible dataset pipeline

This chapter clearly demonstrates that **persistence, adaptability, and technical problem-solving** were just as essential as theoretical knowledge in achieving the success of this project.

More importantly, it proves that the AMTE SecAI system is not only a research concept, but a **practical, scalable, and real-world applicable cybersecurity solution.**